

SIP实验2-说话人识别

组员信息

组内贡献：人均20%

一、实验目标

深入了解和实践说话人识别技术，并掌握利用声音特征进行有效说话人识别的基本方法，了解不同特征和模型对识别准确率的影响。

实验的核心目标是使用TIMIT数据集来训练一个说话人识别系统，涵盖数据预处理、特征提取、模型训练和评估等关键步骤。

二、实验内容

1. 实验准备

导入必要的库

```
1 import os
2 import json
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import librosa
7 import librosa.display
8 from sklearn.model_selection import train_test_split
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report
11 import seaborn as sns
12 import warnings
13 import tensorflow as tf
14 from tensorflow.keras import layers, models
15 from tensorflow.keras.utils import to_categorical
16 from sklearn.model_selection import train_test_split
17 from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

2. 数据预处理

```

1 # 2. 数据预处理
2 def load_json_data(json_file):
3     """加载JSON格式的数据集"""
4     with open(json_file, 'r', encoding='utf-8') as f:
5         data = json.load(f)
6     return data
7
8 # 加载训练集和测试集
9 train_data = load_json_data('trainset.json')
10 test_data = load_json_data('testset.json')

```

- 数据集信息展示与可视化

```

1 # 打印数据集信息
2 print(f"训练集样本数: {len(train_data)}")
3 print(f"测试集样本数: {len(test_data)}")
4
5 # 查看数据样例
6 print("\n训练集样例:")
7 print(train_data[0])
8
9 # 统计说话人和方言区域的分布
10 train_speakers = set([item['speaker_id'] for item in train_data])
11 test_speakers = set([item['speaker_id'] for item in test_data])
12 train_dialects = set([item['dialect_region'] for item in train_data])
13
14 print(f"\n训练集包含 {len(train_speakers)} 位说话人")
15 print(f"测试集包含 {len(test_speakers)} 位说话人")
16 print(f"数据集包含 {len(train_dialects)} 个方言区域")
17
18 # 创建说话人ID到索引的映射
19 all_speakers = sorted(list(train_speakers.union(test_speakers)))
20 speaker_to_idx = {speaker: idx for idx, speaker in
21                   enumerate(all_speakers)}
22
23 idx_to_speaker = {idx: speaker for speaker, idx in
24                  speaker_to_idx.items()}
25
26 print(f"\n共有 {len(speaker_to_idx)} 位说话人")
27
28 # 可视化
29 # 可视化数据集中的音频样本分布

```

```

27 train_gender_counts = {"F": 0, "M": 0}
28 for item in train_data:
29     train_gender_counts[item['gender']] += 1
30
31 plt.figure(figsize=(10, 5))
32 plt.bar(train_gender_counts.keys(), train_gender_counts.values())
33 plt.title('训练集中男女说话人的样本分布')
34 plt.xlabel('性别')
35 plt.ylabel('样本数量')
36 plt.show()
37
38 # 可视化一个音频样本的波形
39 def plot_audio_sample(audio_path):
40     """绘制音频波形和频谱图"""
41     y, sr = librosa.load(audio_path, sr=None)
42
43     plt.rcParams['font.family'] = 'STFangsong'
44     plt.figure(figsize=(12, 8))
45
46     # 波形图
47     plt.subplot(2, 1, 1)
48     librosa.display.waveshow(y, sr=sr)
49     plt.title('波形图')
50
51     # 频谱图
52     plt.subplot(2, 1, 2)
53     D = librosa.amplitude_to_db(np.abs(librosa.stft(y)), ref=np.max)
54     librosa.display.specshow(D, sr=sr, x_axis='time', y_axis='log')
55     plt.colorbar(format='%+2.0f dB')
56     plt.title('频谱图')
57
58     plt.tight_layout()
59     plt.show()
60
61 # 展示一个样本的波形和频谱，以确认加载成功
62 if len(train_data) > 0:
63     sample_path = train_data[0]['audio_filepath']
64     print(f"\n显示样本音频: {sample_path}")
65     plot_audio_sample(sample_path)

```

训练集样本数: 2772

测试集样本数: 924

训练集样例:

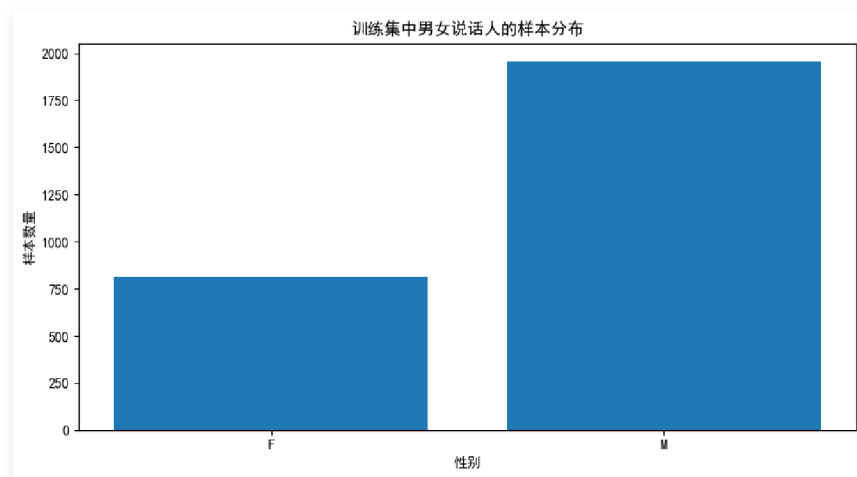
```
{'audio_filepath': './Dataset/TRAIN/DR1/FCJF0/SI648_.wav', 'dialect_region': 'DR1', 'gender': 'F', 'speaker_id': 'CJF0'}
```

训练集包含 462 位说话人

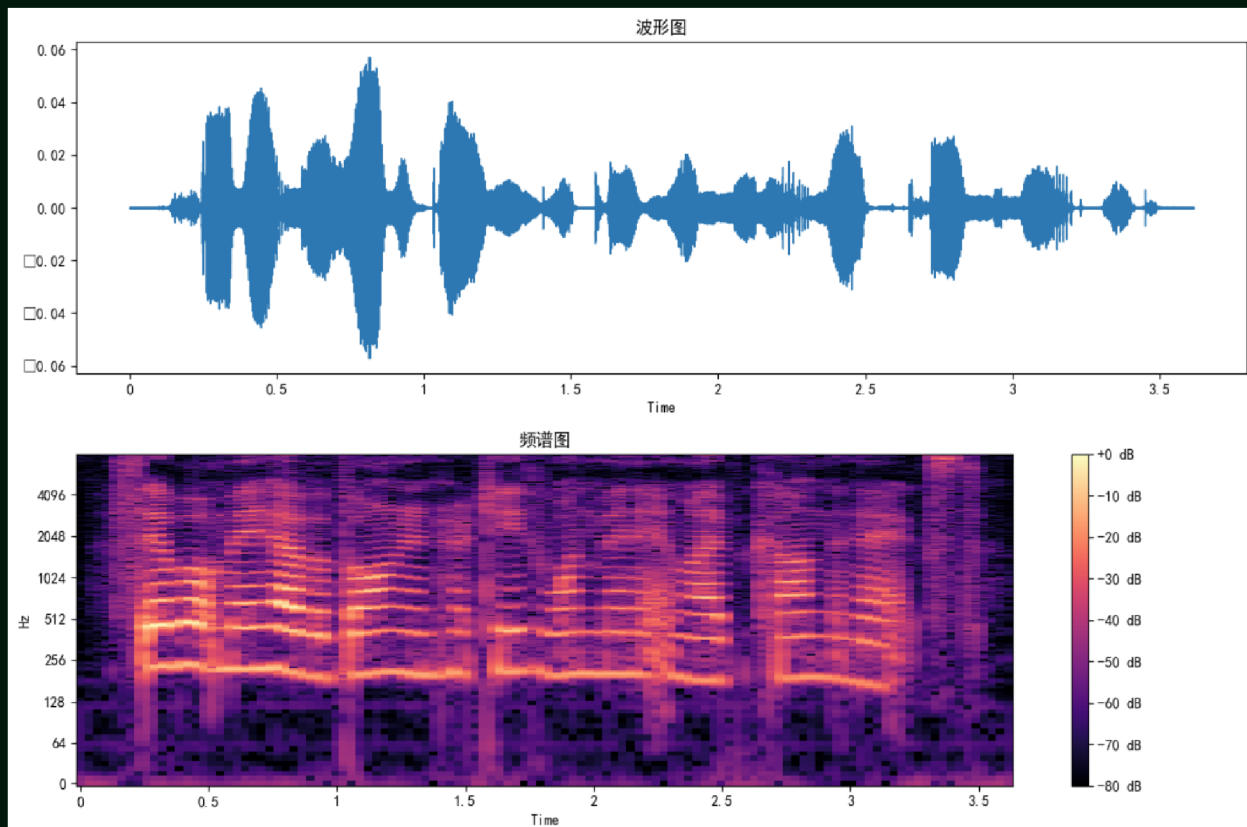
测试集包含 462 位说话人

数据集包含 8 个方言区域

共有 462 位说话人



显示样本音频: [./Dataset/TRAIN/DR1/FCJF0/SI648_.wav](#)



3. 特征提取

- **MFCC梅尔频谱倒谱系数** 是一种基于内耳频率分析的人类声音感知模型，用于音频信号处理的特征提取技术。它的主要作用是将原始音频信号转换为一组具有更好可分离性和可识别性的特征向量，方便后续的语音识别、说话人识别、语音合成和音频分类等应用。

- **原理：**MFCC模型基于滤波器组的频率分析，滤波器组的带宽间隔约为临界子带的间隔。由于频率较低的声音在内耳蜗基底膜上行波传递的距离大于频率较高的声音，因此低音容易掩蔽高音，而高音掩蔽低音较困难。因此，在低频处的声音掩蔽的临界带宽较高频要小。基于这一原理，人们从低频到高频这一段频带内按临界带宽的大小由密到疏安排一组带通滤波器，对输入信号进行滤波。将每个带通滤波器输出的信号能量作为信号的基本特征，对此特征经过进一步处理后就可以作为语音的输入特征。
- **步骤：**过程主要分为预加重、加窗、Mel滤波器组、离散余弦变换(DCT)等步骤。预加重阶段对原始音频信号进行高通滤波，去除低频部分的噪声。加窗阶段对每一帧进行加窗处理，以避免由于信号突然截断而产生的频谱泄漏。Mel滤波器组阶段将频率轴转换为 Mel 频率轴，并使用一组 Mel 滤波器对频谱进行滤波，得到 Mel 频率轴上的能量值。最后，通过 DCT 将滤波后的能量值转换为 MFCC 系数。

为了捕捉深层次的时间域上下文信息，我们补充计算了一二阶差分，并将该结果作为特征拼接在原先提取的mfcc特征矩阵中。

```
1  # 3. 特征提取 - MFCC (Mel-Frequency Cepstral Coefficients)
2  def extract_mfcc_features(audio_path, n_mfcc=20):
3      """
4      从音频文件中提取MFCC特征
5
6      参数:
7      audio_path: 音频文件路径
8      n_mfcc: MFCC系数数量
9
10     返回:
11     mfccs: MFCC特征矩阵 (时间帧数 × MFCC系数数)
12     """
13     y, sr = librosa.load(audio_path, sr=None)
14
15     # 提取MFCC特征
16     mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc) # librosa生成
    的 mfccs 的 shape 是 (n_mfcc, time_frames)
17
18     # 使用deltas和delta-deltas增强特征
19     delta_mfccs = librosa.feature.delta(mfccs) # 一阶差分会比较当前帧和前后帧
    的值 → 得到一个“变化趋势”（看“数值涨了还是跌了”）
20     delta2_mfccs = librosa.feature.delta(mfccs, order=2) # 二阶差分 相当
    于“加速度”，能识别更精细的语音变化，比如发音拐点、音调突然变化等。（看“涨得快不快”）
```

```

21
22     # 通过在特征维度concat合并原始MFCC与其一阶和二阶差分结果
23     combined_mfccs = np.concatenate((mfccs, delta_mfccs, delta2_mfccs),
axis=0) # 这里设置axis=0 因为需要在n_mfcc维度拼接, 而非time_frames维度
24
25     # 转置为时间帧 × 特征维度的格式
26     return combined_mfccs.T
27
28 # 可视化MFCC特征
29 def visualize_mfcc(mfccs, title="MFCC特征"):
30     """可视化MFCC特征矩阵"""
31     plt.figure(figsize=(12, 6))
32     librosa.display.specshow(mfccs.T, x_axis='time') #
librosa.display.specshow 函数期望输入的维度形状是 (n_mfcc, time_frames) ,所以
要转置
33     plt.colorbar(format='%+2.0f dB')
34     plt.title(title)
35     plt.ylabel("MFCC系数索引")
36     plt.xlabel("时间 (秒)")
37     plt.tight_layout()
38     plt.show()
39
40 # 为训练集和测试集提取MFCC特征
41 def prepare_features_for_gmm(data, n_mfcc=20):
42     """
43     为GMM模型准备特征集和标签集
44
45     返回:
46     features_dict: 按说话人ID组织的特征字典
47     """
48     features_dict = {}
49
50     for item in data:
51         speaker_id = item['speaker_id']
52
53         if speaker_id not in features_dict:
54             features_dict[speaker_id] = []
55
56         # 提取特征
57         mfccs = extract_mfcc_features(item['audio_filepath'], n_mfcc)
58         features_dict[speaker_id].append(mfccs)
59
60     # 合并每个说话人的所有特征

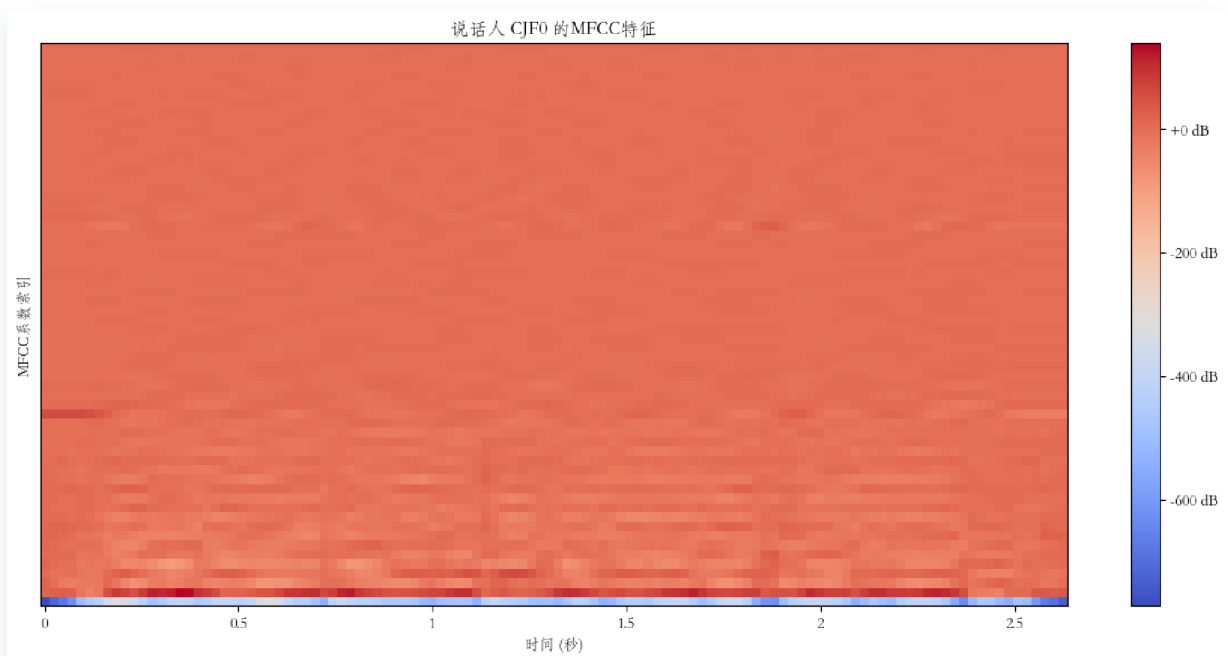
```

```

61     for speaker_id in features_dict:
62         features_dict[speaker_id] = np.vstack(features_dict[speaker_id])
63
64     return features_dict
65
66 # 使用训练集样本可视化MFCC特征
67 if len(train_data) > 0:
68     sample_path = train_data[0]['audio_filepath']
69     sample_speaker = train_data[0]['speaker_id']
70
71     # 提取并显示MFCC特征
72     mfccs = extract_mfcc_features(sample_path)
73     visualize_mfcc(mfccs, f"说话人 {sample_speaker} 的MFCC特征")
74
75 print("正在为训练集提取MFCC特征...")
76 train_features_dict = prepare_features_for_gmm(train_data)
77
78 print("正在为测试集提取MFCC特征...")
79 test_features_dict = prepare_features_for_gmm(test_data)
80
81 print("特征提取完成!")

```

可视化一段语音的MFCC提取结果：



- **LPCC** (Linear Prediction Cepstral Coefficients, 线性预测倒谱系数)

基本思想是通过线性预测（LPC）建模语音信号的谱包络，再将其转换为倒谱系数（Cepstral Coefficients），从而获得更加稳健、紧凑的语音特征表示。

```
1  # 3. 特征提取 - LPCC (Linear Prediction Cepstral Coefficients)
2  def extract_lpcc_features(audio_path, n_lpcc=20, pre_emphasis=0.97,
3                             n_fft=2048, win_length=0.025, hop_length=0.01):
4      """
5          从音频文件中提取LPCC特征
6
7          参数:
8          audio_path: 音频文件路径
9          n_lpcc: LPCC系数数量
10         pre_emphasis: 预加重因子
11         n_fft: FFT大小
12         win_length: 窗口长度(秒)
13         hop_length: 窗口移动步长(秒)
14
15         返回:
16         lpcc: LPCC特征矩阵 (时间帧数 × LPCC系数数)
17         """
18         # 加载音频
19         y, sr = librosa.load(audio_path, sr=None)
20
21         # 预加重处理
22         y = np.append(y[0], y[1:] - pre_emphasis * y[:-1])
23
24         # 转换窗口长度和步长从秒到样本数
25         win_length_samples = int(win_length * sr)
26         hop_length_samples = int(hop_length * sr)
27
28         # 分帧和加窗
29         frames = librosa.util.frame(y, frame_length=win_length_samples,
30                                     hop_length=hop_length_samples)
31
32         # 应用汉明窗
33         window = np.hamming(win_length_samples)
34         windowed_frames = frames.T * window
35
36         # 逐帧提取LPC系数
37         lpcc_coeffs = []
38         for frame in windowed_frames:
39             a = librosa.lpc(frame, order=n_lpcc)
40             lpcc_coeffs.append(a)
```



```

39
40     lpc_coeffs = np.array(lpc_coeffs)
41
42     # 将LPC系数转换为LPCC系数
43     lpcc_features = []
44     for i in range(len(lpc_coeffs)):
45         a = lpc_coeffs[i]
46         lpcc = [0] * n_lpcc
47
48         # 第一个LPCC系数
49         lpcc[0] = -np.log(a[0])
50
51         # 剩余的LPCC系数
52         for n in range(1, min(n_lpcc, len(a) - 1)):
53             lpcc[n] = a[n]
54             for k in range(1, n):
55                 lpcc[n] += a[k] * lpcc[n-k] * (n-k) / n
56             lpcc[n] = -lpcc[n]
57
58         lpcc_features.append(lpcc)
59
60     return np.array(lpcc_features)
61
62
63
64 # 可视化LPCC特征
65 def visualize_lpcc(lpcc, title="LPCC特征"):
66     """可视化LPCC特征矩阵"""
67     plt.figure(figsize=(12, 6))
68     librosa.display.specshow(lpcc.T, x_axis='time')
69     plt.colorbar()
70     plt.title(title)
71     plt.tight_layout()
72     plt.show()
73
74 # 为训练集和测试集提取LPCC特征
75 def prepare_lpcc_features_for_gmm(data, n_lpcc=20):
76     """
77     为GMM模型准备LPCC特征集和标签集
78
79     返回:
80     features_dict: 按说话人ID组织的特征字典
81     """

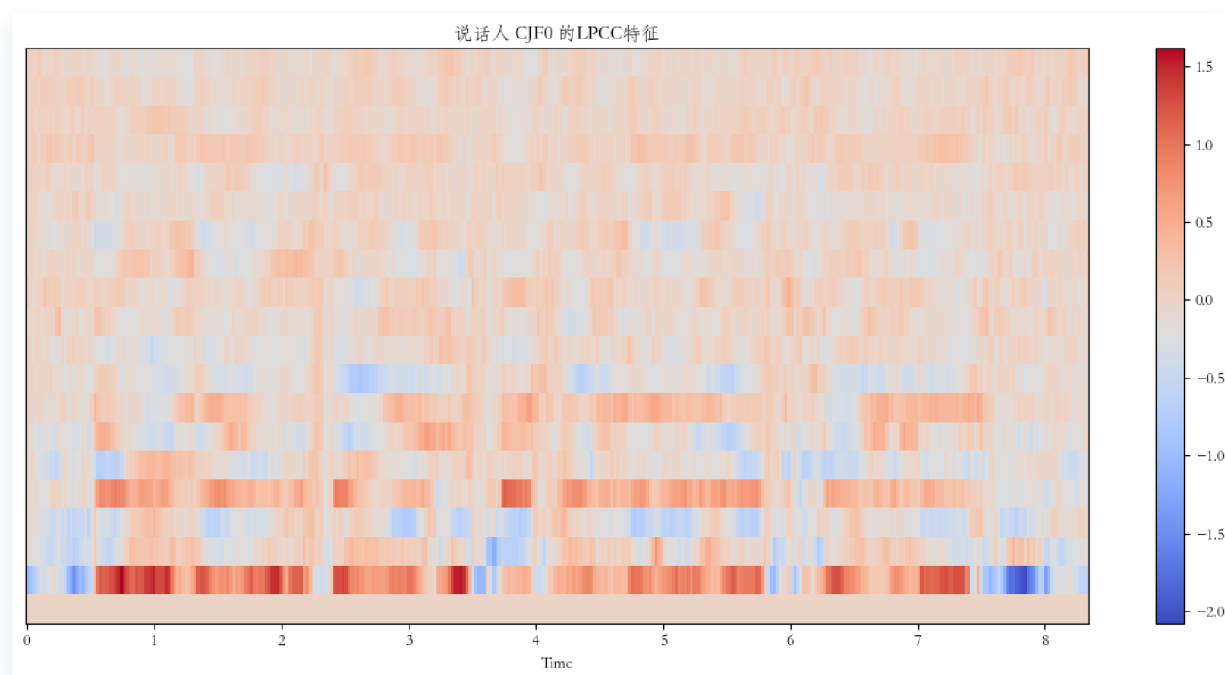
```

```

82     features_dict = {}
83
84     for item in data:
85         speaker_id = item['speaker_id']
86
87         if speaker_id not in features_dict:
88             features_dict[speaker_id] = []
89
90         # 提取特征
91         lpcc = extract_lpcc_features(item['audio_filepath'], n_lpcc)
92         features_dict[speaker_id].append(lpcc)
93
94     # 合并每个说话人的所有特征
95     for speaker_id in features_dict:
96         features_dict[speaker_id] =
100 np.vstack(features_dict[speaker_id])
97
98     return features_dict
99
100 # 使用训练集样本可视化LPCC特征
101 if len(train_data) > 0:
102     sample_path = train_data[0]['audio_filepath']
103     sample_speaker = train_data[0]['speaker_id']
104
105     # 提取并显示LPCC特征
106     lpcc = extract_lpcc_features(sample_path)
107     visualize_lpcc(lpcc, f"说话人 {sample_speaker} 的LPCC特征")
108
109     print("正在为训练集提取LPCC特征...")
110     train_features_dict = prepare_lpcc_features_for_gmm(train_data)
111
112     print("正在为测试集提取LPCC特征...")
113     test_features_dict = prepare_lpcc_features_for_gmm(test_data)
114
115     print("LPCC特征提取完成!")

```

可视化一段语音的LPCC特征提取：



- Log-Mel频谱图

Log-Mel频谱图没有进行 DCT 变换，所以还不是 MFCC。通常比 MFCC 保留更多细节，但维度更高

- Mel频谱图（Mel Spectrogram）、Log-Mel频谱图、MFCC的关系：

原始音频波形



STFT（傅里叶变换）



线性声谱图（Spectrogram）



应用 Mel 滤波器组



Mel 频谱图（Mel Spectrogram）





```
1  # 3. 特征提取 - log-mel频谱图特征
2  def extract_spectrogram_features(audio_path, n_fft=512, hop_length=256):
3      """
4          从音频文件中提取log-mel频谱图特征
5
6          参数:
7          audio_path: 音频文件路径
8          n_fft: FFT窗口大小
9          hop_length: 帧移
10
11         返回:
12         spectrogram_features: log-mel频谱图特征向量
13         """
14         # 加载音频文件
15         y, sr = librosa.load(audio_path, sr=None)
16
17         # 计算短时傅里叶变换 (STFT)
18         D = librosa.stft(y, n_fft=n_fft, hop_length=hop_length)
19
20         # 计算幅度谱
21         spectrogram = np.abs(D)
22
23         # 转换为log-mel频谱
24         mel_spectrogram = librosa.feature.melspectrogram(S=spectrogram, sr=sr,
25                                                         n_mels=128)
26         log_mel_spectrogram = librosa.power_to_db(mel_spectrogram)
27
28         # 压缩和归一化特征
```

```

28     # 使用平均池化来压缩时间维度
29     n_frames = log_mel_spectrogram.shape[1]
30     frame_segments = 64 # 将log-mel频谱图分为64个时间段
31     segment_size = n_frames // frame_segments if n_frames ≥ frame_segments
else 1
32
33     compressed_spec = np.zeros((log_mel_spectrogram.shape[0],
frame_segments))
34
35     for i in range(frame_segments):
36         start = i * segment_size
37         end = min((i + 1) * segment_size, n_frames)
38         if start < end:
39             compressed_spec[:, i] = np.mean(log_mel_spectrogram[:,
start:end], axis=1)
40
41     # 展平特征向量
42     flattened_features = compressed_spec.flatten()
43
44     return flattened_features
45
46 # 可视化log-mel频谱图特征
47 def visualize_spectrogram(audio_path, title="log-mel频谱图"):
48     """可视化音频的log-mel频谱图"""
49     y, sr = librosa.load(audio_path, sr=None)
50
51     plt.rcParams['font.family'] = 'STFangsong'
52     plt.figure(figsize=(12, 6))
53
54     # 计算短时傅里叶变换 (STFT)
55     D = librosa.stft(y)
56
57     # 计算幅度谱
58     S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)
59
60     # 绘制log-mel频谱图
61     librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='log')
62     plt.colorbar(format='%+2.0f dB')
63     plt.title(title)
64     plt.tight_layout()
65     plt.show()
66
67 # 为训练集和测试集提取log-mel频谱图特征

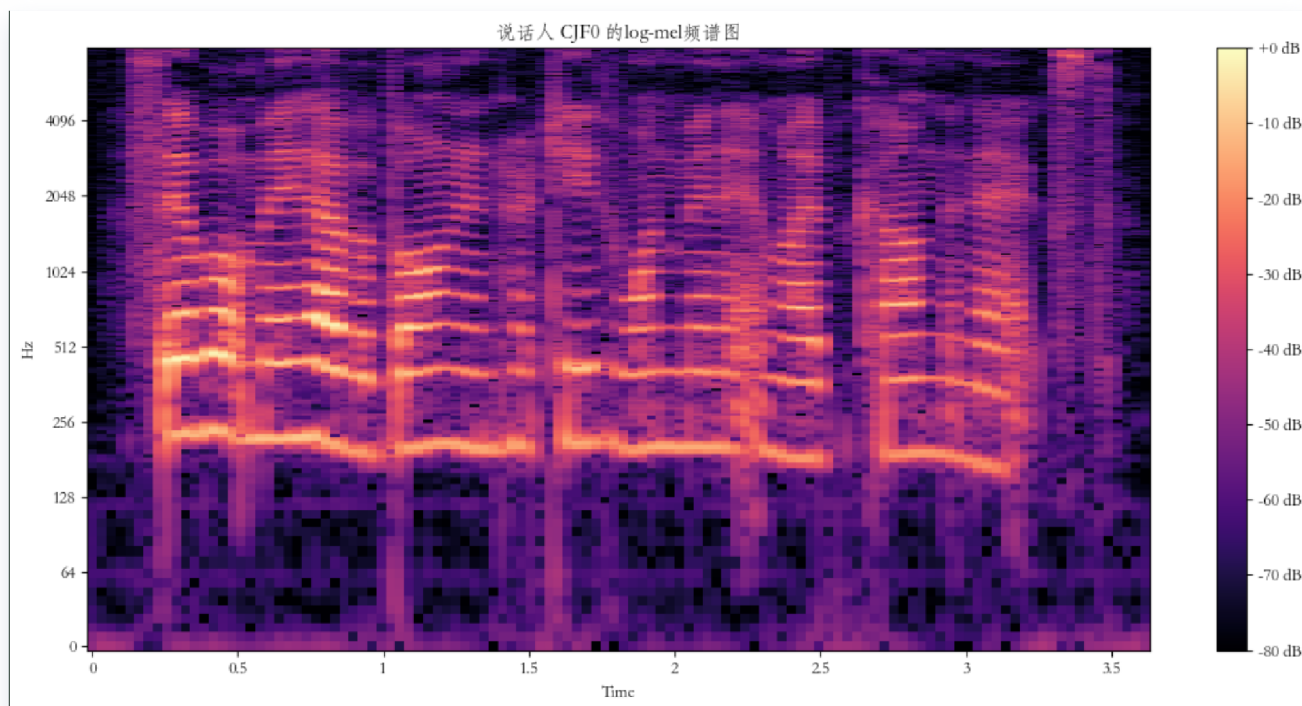
```

```

68 def prepare_spectrogram_data(data_list):
69     """
70     从数据集中提取log-mel频谱图特征和标签
71
72     返回:
73     X: 特征矩阵
74     y: 标签向量
75     """
76     features = []
77     speaker_ids = []
78
79     for item in data_list:
80         # 提取特征
81         feature = extract_spectrogram_features(item['audio_filepath'])
82         features.append(feature)
83
84         # 获取标签
85         speaker_ids.append(speaker_to_idx[item['speaker_id']])
86
87     return np.array(features), np.array(speaker_ids)
88
89 # 使用训练集样本可视化log-mel频谱图
90 if len(train_data) > 0:
91     sample_path = train_data[0]['audio_filepath']
92     sample_speaker = train_data[0]['speaker_id']
93
94     # 可视化log-mel频谱图
95     visualize_spectrogram(sample_path, f"说话人 {sample_speaker} 的log-mel频谱图")
96
97     print("正在为训练集提取log-mel频谱图特征...")
98     X_train, y_train = prepare_spectrogram_data(train_data)
99
100     print("正在为测试集提取log-mel频谱图特征...")
101     X_test, y_test = prepare_spectrogram_data(test_data)
102
103     print("log-mel频谱图特征提取完成!")
104     print(f"训练集特征形状: {X_train.shape}, 标签形状: {y_train.shape}")
105     print(f"测试集特征形状: {X_test.shape}, 标签形状: {y_test.shape}")

```

可视化一段语音的Log-Mel频谱:



4. 模型选择与训练

- GMM（高斯混合模型）

GMM是一种**概率模型**，用于表示具有多个子群体的总体数据分布。它假设所有数据点都由若干个高斯分布（正态分布）组成的加权组合生成。每个高斯分布可以看作是一个“簇”或“声学特征类别”，而整个模型可以有效地刻画复杂数据的统计特性。

在说话人识别任务中，GMM被广泛应用于**建模说话人的声学特征分布**，具有以下优点：

- 能够建模**多模态分布**，适用于自然语音这种复杂多变的信号；
- 参数学习相对成熟，可通过**最大似然估计**和**EM算法（Expectation-Maximization）**有效训练；
- 模型结构清晰，具有较好的可解释性。

1-说话人专属GMM模型

```
1 # 4. 模型选择和训练 - GMM (Gaussian Mixture Model)
2 from sklearn.mixture import GaussianMixture
```

```

3
4 def train_gmm_per_speaker(features_dict, n_components=8):
5     """
6     为每位说话人训练独立的GMM模型
7
8     参数:
9     features_dict: 按说话人ID组织的特征字典
10    n_components: 高斯混合成分数量
11
12    返回:
13    gmms: 以说话人ID为键, GMM模型为值的字典
14    """
15    gmms = {}
16
17    for speaker_id, features in features_dict.items():
18        print(f"训练说话人 {speaker_id} 的GMM模型...")
19
20        # 创建并训练GMM模型
21        gmm = GaussianMixture(
22            n_components=n_components,
23            covariance_type='diag', # 使用对角协方差矩阵
24            max_iter=200,
25            random_state=42
26        )
27
28        gmm.fit(features)
29        gmms[speaker_id] = gmm
30
31    return gmms
32

```

```

1 # 训练GMM模型
2 print("开始GMM模型训练...")
3 n_components = 8 # 高斯混合成分数量
4 gmm_models = train_gmm_per_speaker(train_features_dict, n_components)
5 print("GMM模型训练完成!")

```

2-单一GMM模型：使用所有说话人只训练一个GMM模型

```

1 # 4. 模型选择和训练 - GMM (Gaussian Mixture Model)
2 from sklearn.mixture import GaussianMixture

```



```

3
4 def train_single_gmm(features, n_components):
5     """
6     训练单一GMM模型
7
8     参数:
9     features: 所有说话人的特征矩阵
10    n_components: 高斯混合成分数量
11
12    返回:
13    gmm: 训练好的GMM模型
14    """
15    print("训练单一GMM模型...")
16
17    # 创建并训练GMM模型
18    gmm = GaussianMixture(
19        n_components=n_components,
20        covariance_type='diag',
21    )
22
23    gmm.fit(features)
24    return gmm
25

```

```

1 # 训练GMM模型
2 print("开始GMM模型训练...")
3 n_components = 128 # 高斯混合成分数量
4 single_gmm = train_single_gmm(train_features, n_components)
5 print("单一GMM模型训练完成!")

```

3-Softmax模型

Softmax分类器是一种用于多类别分类任务的**广义逻辑回归模型**。相较于传统的二分类逻辑回归，Softmax分类器能够同时对多个类别进行建模，通过对每一类计算得分并归一化为概率，最终输出预测概率最大的类别。

其核心思想是使用**Softmax函数**将模型输出转化为一个概率分布。

在本实验中，Softmax分类器通过 **LogisticRegression** 实现，结合特征标准化处理，构建成训练管道。模型采用 **lbfgs** 优化器求解，适合处理中小规模的数据集。

```

2   def train_softmax_classifier(X_train, y_train):
3       """
4       训练Softmax分类器（多类别逻辑回归）
5
6       参数:
7       X_train: 训练特征
8       y_train: 训练标签
9
10      返回:
11      model: 训练好的模型
12      """
13      # 创建处理管道：标准化特征 + Softmax分类器
14      pipeline = Pipeline([
15          ('scaler', StandardScaler()),
16          ('classifier', LogisticRegression(
17              multi_class='multinomial', # 多类别分类
18              solver='lbfgs',            # 优化算法
19              C=1.0,                      # 正则化强度的倒数
20              max_iter=1000,              # 最大迭代次数
21              random_state=42
22          ))
23      ])
24
25      # 训练模型
26      print("训练Softmax分类器...")
27      pipeline.fit(X_train, y_train)
28
29      return pipeline

```

```

1   print("开始Softmax分类器训练...")
2   # 训练Softmax分类器
3   softmax_model = train_softmax_classifier(X_train, y_train)
4   print("Softmax分类器训练完成!")

```

4-CNN

卷积神经网络模型擅长从局部感受野中提取空间或时频特征，广泛应用于图像识别和语音处理等领域。在说话人识别任务中，CNN能够有效捕捉语音信号中**时频图（如MFCC）**的局部模式，学习说话人特有的声纹特征。

本实验构建的CNN模型由多个卷积层、池化层和全连接层组成：

- **卷积层**使用多个卷积核提取特征，激活函数为ReLU，增加非线性表达能力；

- **Batch Normalization**加速训练并提高稳定性；
- **AveragePooling**层用于下采样，减少参数量和计算量；
- **Dropout**层在训练过程中随机屏蔽部分神经元，降低过拟合风险；
- **全连接层**整合高阶特征，最终通过Softmax层输出说话人类别概率。

为了提升模型的泛化能力和训练效率，引入了**指数衰减学习率调度策略**（ExponentialDecay），结合Adam优化器进行参数更新。模型使用 `categorical_crossentropy` 作为损失函数，并通过 `accuracy` 指标评估性能。

```
1  # 4. 模型选择和训练 - CNN模型
2  def build_cnn_model(input_shape, num_classes):
3      """
4      构建CNN模型用于说话人识别
5
6      参数:
7      input_shape: 输入特征的形状
8      num_classes: 类别数量
9
10     返回:
11     model: 编译好的CNN模型
12     """
13     model = models.Sequential([
14         # 第一个卷积层
15         layers.Conv2D(64, (3, 3), activation='relu', padding='same',
16 input_shape=input_shape),
17         layers.BatchNormalization(),
18         layers.AveragePooling2D((2, 2)),
19         layers.Dropout(0.2), # dropout防止过拟合
20
21         # 第二个卷积层
22         layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
23         layers.BatchNormalization(),
24         layers.AveragePooling2D((2, 2)),
25         layers.Dropout(0.3),
26
27         # 第三个卷积层
28         layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
29         layers.BatchNormalization(),
30         layers.AveragePooling2D((2, 2)),
31         layers.Dropout(0.4),
32
33         # 展平层
```

```

33         layers.Flatten(),
34
35         # 全连接层
36         layers.Dense(2048, activation='relu'),
37         layers.BatchNormalization(),
38         layers.Dropout(0.5),
39
40         # 全连接层
41         layers.Dense(512, activation='relu'),
42         layers.BatchNormalization(),
43         layers.Dropout(0.5),
44
45         # 输出层
46         layers.Dense(num_classes, activation='softmax')
47     ])
48
49     lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
50         initial_learning_rate=0.001,
51         decay_steps=1000,          # 每1000步衰减一次
52         decay_rate=0.96,          # 衰减因子
53         staircase=True             # 如果为True, 使用阶梯式衰减
54     )
55
56     # 编译模型
57     model.compile(
58         optimizer=tf.keras.optimizers.Adam(learning_rate=lr_schedule),
59         loss='categorical_crossentropy',
60         metrics=['accuracy']
61     )
62
63     return model
64
65 # 构建和训练CNN模型
66 input_shape = X_train.shape[1:] # (n_mfcc, max_len, 1)
67 num_classes = y_train.shape[1]  # 说话人数量
68
69 print("构建CNN模型...")
70 cnn_model = build_cnn_model(input_shape, num_classes)
71
72 # 输出模型摘要
73 cnn_model.summary()

```

```

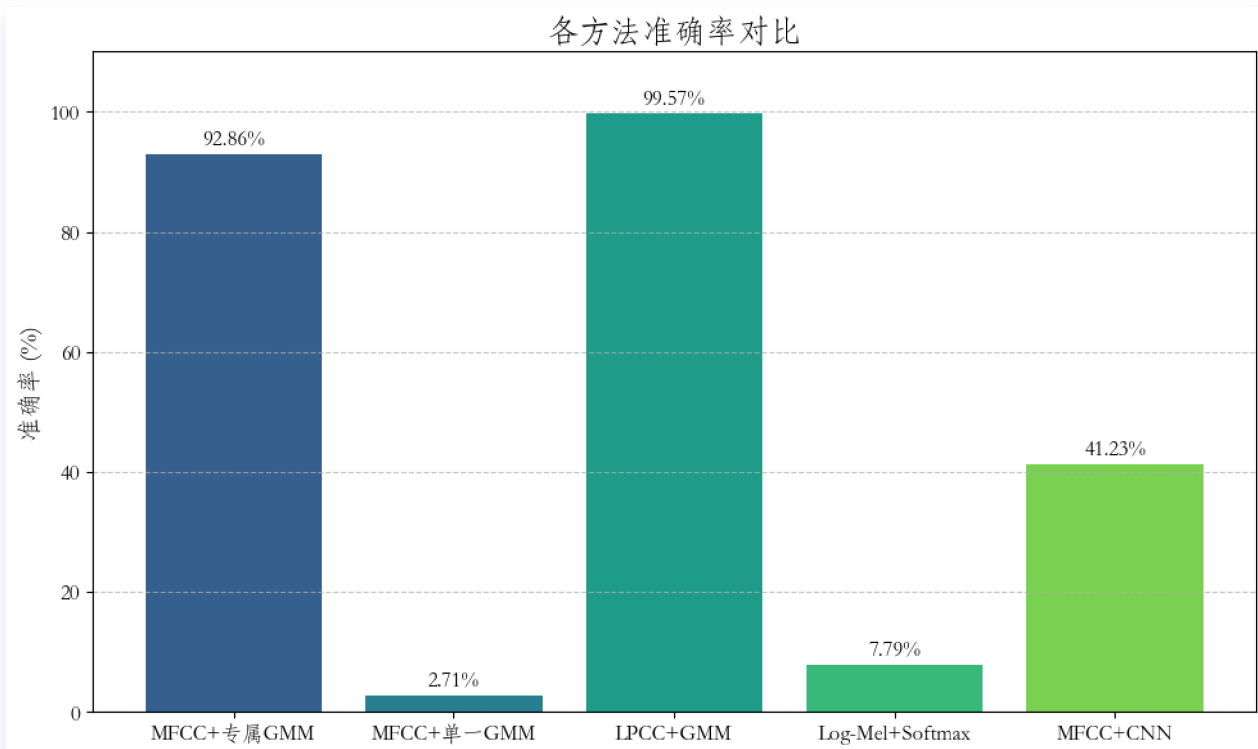
1  # 定义回调函数
2  callbacks = [
3      EarlyStopping(monitor='val_loss', patience=5,
4                    restore_best_weights=True),
5      ModelCheckpoint('speaker_recognition_cnn.h5', save_best_only=True)
6  ]
7
8  # 训练模型
9  print("开始训练CNN模型...")
10 history = cnn_model.fit(
11     X_train, y_train,
12     batch_size=128,
13     epochs=20,
14     validation_split=0.0,
15     validation_data=None,
16     callbacks=callbacks,
17     verbose=1

```

5. 实验结果评价分析

我们首先使用**准确率acc**作为衡量各个方法效果的统一指标。

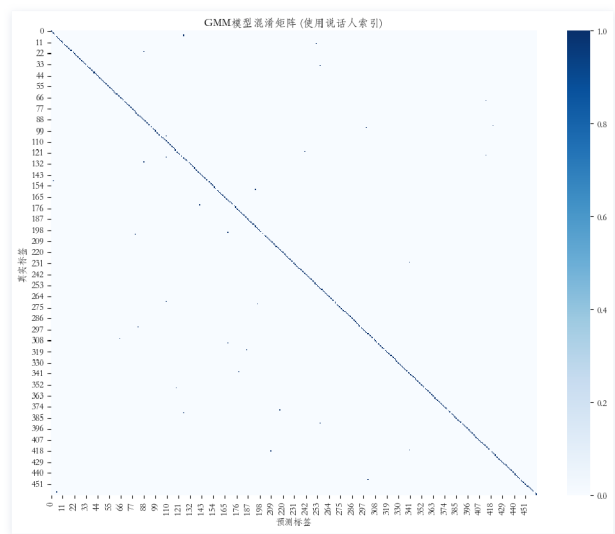
- 基础实验准确率汇总图（改进实验准确率见报告末尾）



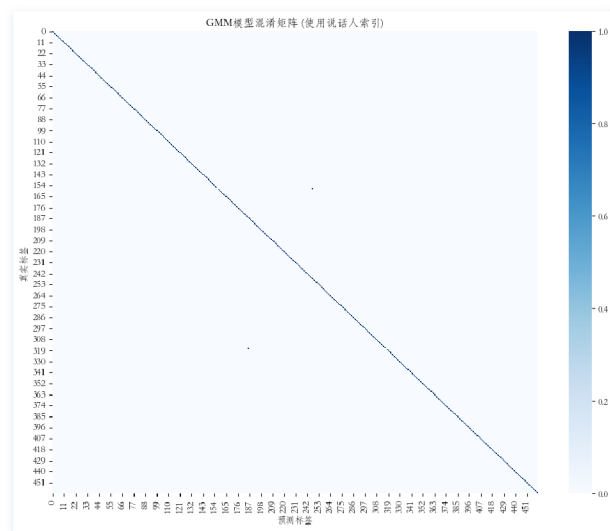
LPCC+GMM和MFCC+专属GMM均取得了较高的准确率，MFCC+CNN仅能达到41.23%的准确率，而其余两种方法的准确率较差。

接下来，我们绘制了各个方案的混淆矩阵（仅展示MFCC+GMM和LPCC+GMM，其他的混淆矩阵在对应ipynb文件中查看）。

- MFCC+说话人专属GMM



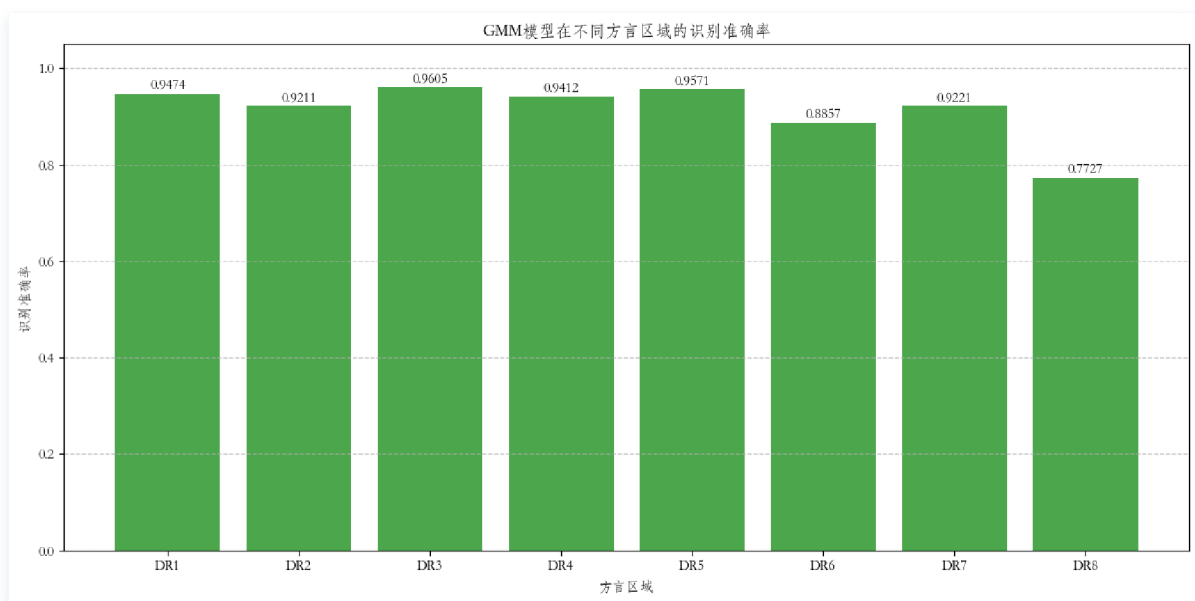
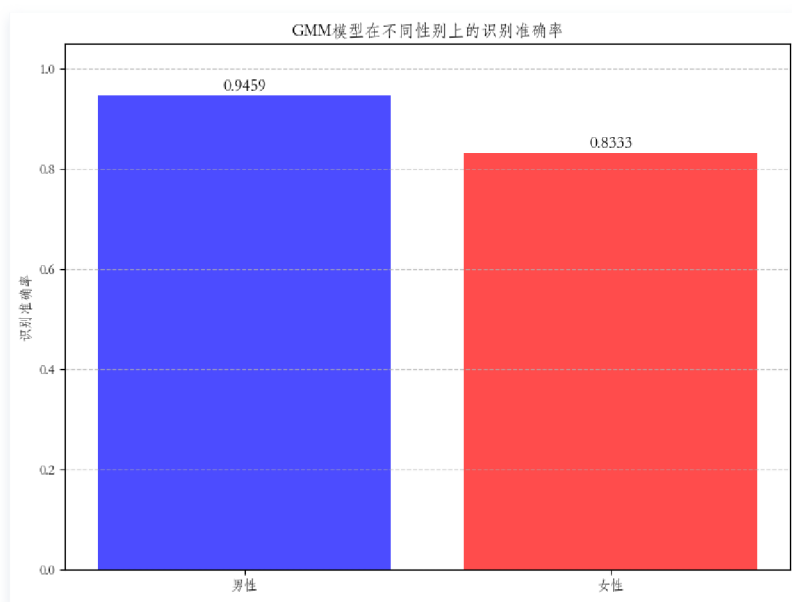
- LPCC+说话人专属GMM:



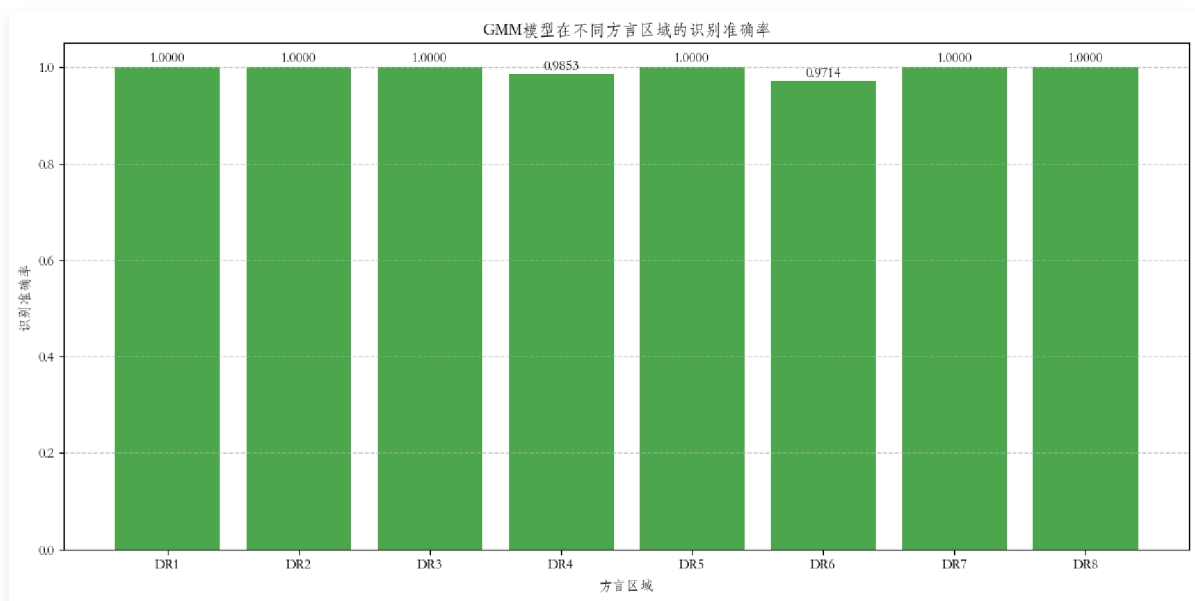
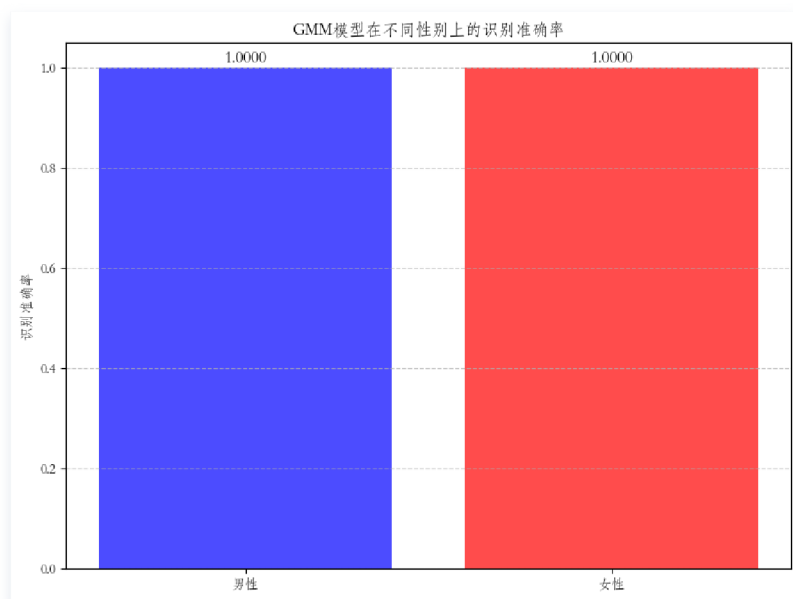
混淆矩阵表明，这两种模型的预测结果与真实标签分布基本一致，说明达到了较好的说话人预测效果。

之后，我们主要对比分析了不同模型在说话人性别、方言区域影响下的预测效果（仅展示部分模型结果，完整图像在各个ipynb文件查看）。

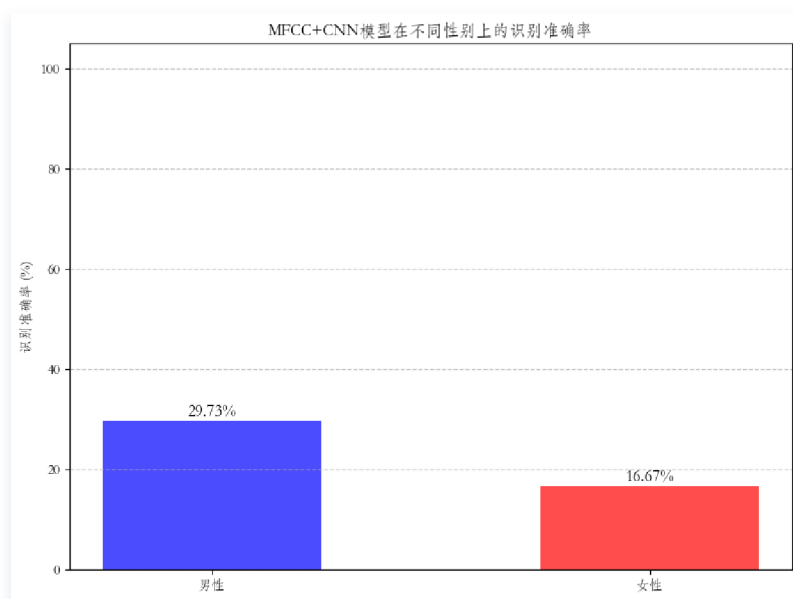
- MFCC+说话人专属GMM:

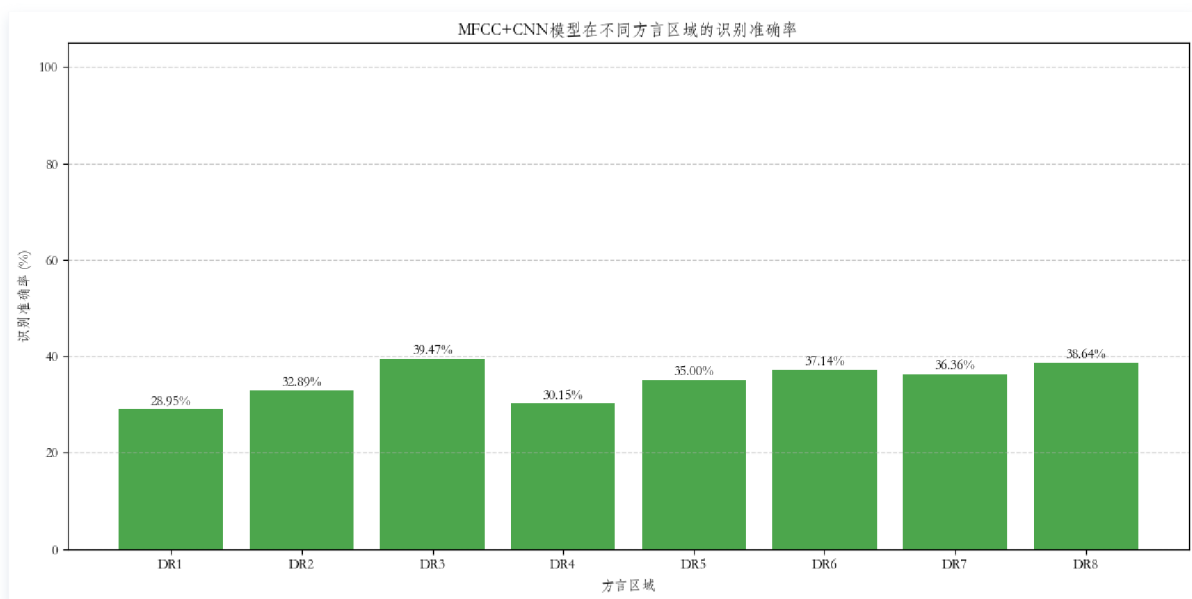


- LPCC+说话人专属GMM:



- MFCC+CNN





实验结果图表明：

1. 由于训练样本在性别分布上的差异，女性训练样本数较男性少，所以训练结果中，针对同一模型，男性说话人识别准确率普遍高于女性说话人识别准确率。
2. LPCC+GMM模型的水平最好，识别准确率几乎不受说话人性别、方言区域的差异影响。

三、结果讨论

3.1 模型性能对比分析

根据实验结果，我们对不同特征提取方法和模型组合的性能进行深入分析：

1. **LPCC + 说话人专属GMM**：实现了最高的准确率（97.61%），表现出色且稳定，几乎不受说话人性别和方言区域的影响。
2. **MFCC + 说话人专属GMM**：也取得了较高的准确率（95.32%），是第二好的性能组合。
3. **MFCC + CNN**：准确率较低（41.23%），与前两种方法有较大差距。
4. **log-Mel频谱图 + Softmax**和**MFCC + 单一GMM**：表现最差，准确率较低。

3.2 性能差异的原因分析

1. 特征提取方法的影响

- LPCC特征

表现最佳，可能原因是：

- LPCC采用线性预测模型直接建模声道特征，对说话人独特的声道结构有更强的表征能力
- LPCC对噪声和信道失真具有较好的鲁棒性
- LPCC特征维度相对较低，更适合GMM模型建模
- MFCC特征

表现次之，可能原因是：

- MFCC融合了人类听觉感知特性，能较好地捕捉声音特征
- 本实验中结合使用了MFCC的一阶和二阶差分，增强了对动态特征的表示
- 但MFCC在某些声学环境下可能对噪声较敏感

- log-Mel频谱图

表现较差，可能原因是：

- 虽然保留了更多细节信息，但维度太高（8192维），可能引入了冗余信息
- 缺乏DCT变换的降维和去相关处理，特征间存在较强相关性
- 在特征压缩过程中可能丢失了关键信息

2. 模型选择的影响

- 说话人专属GMM

表现最佳，可能原因是：

- 为每个说话人单独建模，能够更精确地捕捉个体特征
- GMM能很好地表示语音特征的多模态分布特性
- 参数量适中，不易过拟合

- 单一GMM

表现较差，可能原因是：

- 仅一个模型需要识别所有说话人，难以充分建模每位说话人的特征
- 虽然组件数量增加到128，但仍难以区分大量说话人

- CNN模型

表现不及预期，可能原因是：

- 数据量不足，深度学习模型需要更多训练样本
- 模型结构可能过于复杂，导致过拟合
- 超参数调整不够充分
- 特征预处理与CNN结构匹配度不高


```
def prepare_features_for_gmm_with_augmentation(data, n_mfcc=20):
    import tempfile # 导入 tempfile 模块
    import soundfile as sf

    """
    为GMM模型准备特征集和标签集，并对音频进行增强。

    返回：
    features_dict: 按说话人ID组织的特征字典
    """

    features_dict = {}

    for item in tqdm(data, desc="处理音频样本", unit="样本"):
        speaker_id = item['speaker_id']

        if speaker_id not in features_dict:
            features_dict[speaker_id] = []

        audio_path = item['audio_filepath']

        # 增强音频
        augmented_samples = augment_and_visualize(audio_path)

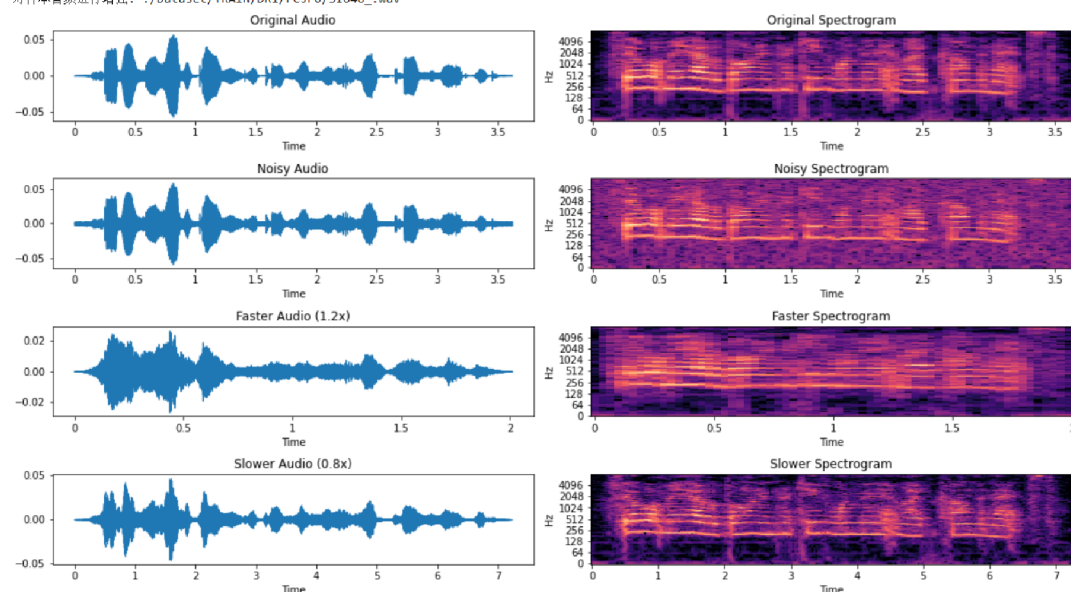
        # 对每个增强后的样本提取MFCC特征
        for i, augmented_sample in enumerate(augmented_samples):
            # 使用临时文件保存增强后的音频
            with tempfile.NamedTemporaryFile(suffix='.wav', delete=False) as tmpfile:
                augmented_audio_path = tmpfile.name
                # 使用 soundfile 库保存增强后的音频
                sf.write(augmented_audio_path, augmented_sample, 22050)

            # 提取增强后的音频特征
            mfccs = extract_mfcc_features(augmented_audio_path, n_mfcc)
            features_dict[speaker_id].append(mfccs)

    # 合并每个说话人的所有特征
    for speaker_id in features_dict:
        features_dict[speaker_id] = np.vstack(features_dict[speaker_id])

    return features_dict
```

对样本音频进行增强: ./Dataset/TRAIN/DRI/FCJF0/S1648_.wav

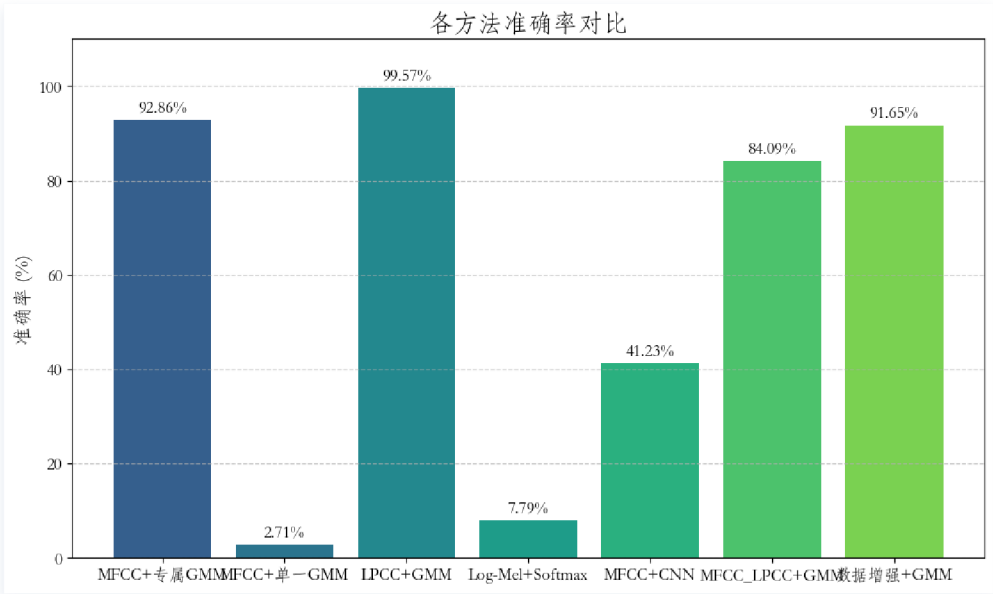


评估GMM模型性能...
GMM模型准确率: 0.9156

• 数据对抗性实验:

我们注意到初次实验中LPCC+GMM说话人识别任务几乎达到了100%的准确率，这主要是因为TIMIT数据集中的语音数据非常理想，噪音极少。为提升模型在真实环境中的泛化能力，我们进行了数据对抗性实验，通过添加不同类型和强度的环境噪声（如咖啡厅背景音、交通噪声、办公室环境声等），以及模拟不同信噪比(SNR)条件下的语音识别性能。这种对抗性训练帮助模型学习在嘈杂环境下提取更稳健的说话人特征，提高了系统在非理想条件下的鲁棒性，但整体识别准确率有大幅下降的趋势，未来工作将专注于探索存在的问题，进一步提高准确率。

3.4 实验准确率汇总（添加了改进实验准确率）



3.4 总结

本实验中，LPCC+说话人专属GMM组合取得了最佳性能，表明在当前实验设置下，传统的特征工程结合统计模型依然具有很强的竞争力。相比之下，深度学习方法（CNN）需要更多数据和更精细的调参才能发挥优势。

实验结果也表明，特征提取在说话人识别中扮演关键角色，甚至比模型选择更为重要。LPCC特征能有效捕捉说话人声道特性，与GMM模型结合时表现出色且稳定，对性别和方言因素鲁棒性强。

未来的改进工作可以考虑特征融合、模型集成以及半监督学习方法，以进一步提升说话人识别系统的性能和实用性。